

Despatch Scale Labels System Documentation

Generated on 07/09/2026, 15:05:20

Despatch Scale Labels — System Documentation

1. Overview

Despatch Scale Labels is a standalone Node.js + Express application that replaces the paper "Packing Labels Request Form" used by the despatch team. It streams live weight from a serial scale indicator, lets an operator key in a packing-labels request (header + one row per carton), computes net weight automatically, stores everything in SQL Server, and produces searchable reports exportable to PDF and Excel.

It runs completely independently of the weighbridge system — its own folder, dependencies, port, and database — though it reuses the same proven serial-scale streaming technique.

Core modules:

- ``app.js``: Express bootstrap and all API routes
- ``db.js``: Knex database client configuration (MSSQL by default; MySQL supported via ``DB_CLIENT``)
- ``db-init.js``: schema bootstrap (idempotent — safe to run on every startup)
- ``lib/scale.js``: serial-port scale streaming engine (COM discovery, auto baud/parity detection, SSE fan-out, stability detection)
- ``lib/labels.js``: validation, net-weight calculation, and persistence for requests and carton lines
- ``lib/settings.js``: crate-weight setting get/save
- ``lib/exporters/pdf.js``, ``lib/exporters/excel.js``: PDF and Excel report generation
- ``public/index.html``, ``public/app.js``, ``public/styles.css``: front-end app (no build step, no login)

2. Key Features and Functions

2.1 Live Scale Integration

- COM port discovery endpoint
- Auto-detection of baud rate/parity by probing a profile list against the selected port
- Live weight streamed to the browser over Server-Sent Events (SSE), with automatic reconnect and idle-disconnect
- Stability detection (median + tolerance across a rolling sample window) — carton weight can only be captured once a reading is stable
- Manual weight entry fallback when no scale is connected, gated behind an explicit UI toggle
- Optional ``SCALE_BAUD_RATE`` / ``SCALE_FORCE_BAUD`` environment override to skip auto-detection for a known indicator

2.2 Packing Labels Request Capture

- Header fields: Labels Requested By*, Labels Applied By, Labels Printed By*, Customer Name*, Customer Reference No., Airway Bill No., Request Date (* mandatory)
- Header is saved first (creating an ``OPEN`` request), then carton lines are added one at a time as they're weighed — each line is persisted immediately, not batched, so an interruption (crash, network blip, power loss) never loses already-weighed cartons
- Carton line fields: Carton No., Product Description, Crate Count, Production Date, Expiry Date,

plus a captured Gross Weight

- Net weight is always computed server-side as ``gross_weight_kg - (crate_count * crate_weight_kg)``, using the crate weight setting in effect at capture time — never trusted from the client
- A request is finalized with ****Complete Request****, which locks it (``COMPLETED``) against further edits
- ****Resume an Open Request****: the New Request tab lists all ``OPEN`` requests and lets an operator resume one (useful for shift handovers); the browser also remembers the in-progress request in ``localStorage`` and auto-resumes it after a page refresh
- Product Description, Production Date, and Expiry Date are deliberately **not** cleared between lines, since they typically repeat across a run of cartons for the same customer/batch; Carton No. and the captured weight always clear, since those are unique per carton

2.3 Crate Weight Setting

- Standard crate tare weight (default 1.8kg) is stored in ``app_settings`` and editable via API
- Each carton line snapshots the crate weight in effect when it was captured, so changing the setting later never rewrites historical net weights

2.4 Reports Module

- Filter requests by date range, customer name, requested-by, and airway bill no.
- Per-request PDF export, mirroring the paper form's header + carton table layout, with gross/net totals
- Per-request Excel export (one row per carton line, header columns repeated)
- Bulk "Export filtered" Excel export of the report list (one row per request, with line count and net weight total)

3. Data Model

Schema is auto-initialized at startup via ``db-init.js`` (creates tables only if missing; safe to re-run):

``despatch_requests`` (header)

- ``id``, ``labels_requested_by``, ``labels_applied_by``, ``labels_printed_by``, ``customer_name``, ``customer_reference_no``, ``airway_bill_no``, ``request_date``, ``status`` (``OPEN`` / ``COMPLETED``), ``created_at``, ``updated_at``

``despatch_lines`` (carton rows)

- ``id``, ``despatch_request_id`` (FK, cascades on delete), ``line_no``, ``carton_no``, ``product_description``, ``gross_weight_kg``, ``crate_count``, ``crate_weight_kg`` (snapshot at capture time), ``net_weight_kg`` (server-computed), ``production_date``, ``expiry_date``, ``created_at``

``app_settings``

- Key/value store; ``crate_settings`` holds `{ crateWeightKg }`

4. API Surface

4.1 Scale

- ``GET /api/get-comport-list``
- ``GET /api/scale/stream/:com`` (Server-Sent Events: ``connected``, ``reading``, ``status``, ``error``, ``heartbeat``)

4.2 Settings

- `GET /api/settings/crate-weight`
- `PUT /api/settings/crate-weight`

4.3 Requests

- `POST /api/requests`
- `GET /api/requests` (filters: `from`, `to`, `customerName`, `requestedBy`, `airwayBillNo`, `status`; pagination: `page`, `pageSize`)
- `GET /api/requests/export/excel` (bulk report export; same filters as above)
- `GET /api/requests/:id`
- `PATCH /api/requests/:id`
- `POST /api/requests/:id/complete`
- `GET /api/requests/:id/pdf`
- `GET /api/requests/:id/excel`

4.4 Carton Lines

- `POST /api/requests/:id/lines`
- `PUT /api/requests/:id/lines/:lineId`
- `DELETE /api/requests/:id/lines/:lineId`

4.5 Health

- `GET /api/health` — reports process and database connectivity status

5. Operational Workflow

1. Operator opens the app (no login required) and, on the New Request tab, either resumes a listed `OPEN` request or starts a new one.
2. Header fields are filled in and saved (**Save Request**), creating an `OPEN` request.
3. Operator selects and connects the scale's COM port; live weight streams to the page with a LIVE/STABLE/OFFLINE badge.
4. For each carton: enter Carton No. and (first time) Product Description/dates, click **Capture Weight from Scale** once the reading is stable (or use manual entry), review the computed net weight, then **Add Line** — the line is saved to the database immediately.
5. Repeat for each carton in the batch; Product Description and dates persist between lines to speed up repetitive entry.
6. Once all cartons are weighed, click **Complete Request** to lock it.
7. On the Reports tab, search/filter past requests and download PDF/Excel per request, or export the filtered list in bulk.

6. Important Configuration

Environment variables (`.env`):

- `PORT` (default `3100`)
- `DB\_CLIENT` (`mssql` by default; `mysql2` also supported)
- `DB\_HOST`, `DB\_PORT`, `DB\_USER`, `DB\_PASSWORD`, `DB\_NAME`
- `SCALE\_BAUD\_RATE` — force a specific baud rate instead of auto-detecting
- `SCALE\_FORCE\_BAUD` (`false` by default) — when `true`, only try `SCALE\_BAUD\_RATE`, skipping the fallback profile list

The target database (e.g. `despatch\_labels`) must already exist on the SQL Server instance; the app creates its own tables inside it automatically. If the database is unreachable at startup, the app still

starts (serving the UI and scale endpoints), but every `/api/requests*` and `/api/settings*` endpoint returns `503` until connectivity is restored.

7. Security Considerations

- No authentication layer — this app is intended for a trusted, single-purpose despatch-floor workstation on the local network, not public exposure
- Keep `.env` out of source control (already covered by `.gitignore`)
- Restrict the SQL Server login used here to least privilege on its own `despatch_labels` database
- Net weight is always recalculated server-side from the stored crate-weight setting; a tampered client payload cannot forge a different net weight

8. Known Limits and Future Enhancements

- No automated tests currently
- No authentication/role separation — if the app is ever exposed beyond a single trusted workstation, add a login layer (the weighbridge system's JWT + role-guard pattern can be reused)
- No line-editing UI yet (`PUT /api/requests/:id/lines/:lineId` exists in the API but isn't wired into the front end) — lines can currently only be added or removed, not edited in place
- Reports are unpaginated beyond a fixed page size cap; add pagination controls in the UI as request volume grows
- `serialport@9` requires a natively compiled binding; see the README for the Windows build-tools note